

Type Casting

Type Casting means converting a value from one data to another.

How to use:

There are 2 kinds:

A. Implicit (Widening) Casting -

Smaller $\rightarrow$ larger type, done automatically by Java:

Ex:

```
int a = 10;  
double b = a;  
S.O.p(b);
```

Order (Smaller $\rightarrow$ large):

byte $\rightarrow$ short $\rightarrow$ int $\rightarrow$ long $\rightarrow$ float $\rightarrow$ double

B. Explicit (Narrowing) Casting -

larger $\rightarrow$ smaller type,

you must do it manually.

Ex:

```
double d = 9.78;  
int i = (int) d;  
S.O.p(i);
```

String $\leftrightarrow$ number conversions (a different but related need)

Ex:

```
String s = "25";  
int num = Integer.parseInt(s);  
int x = 25;  
String s2 = String.valueOf(x);  
(Or)  
String s3 = "" + x;
```

When to use:

Use it when you need to:

- * Store a value in a variable of a different (but compatible) type.
- * Perform a calculation that requires a different type than what you currently have (Ex: decimal division from two "int"s).
- * Convert user input (often read as String) into a number, or vice versa.

Where to use:

Widening - happens naturally in expressions mixing types (int + double $\rightarrow$ result is double)

Narrowing - Whenever you deliberately want to "downgrade" a value - ex: truncating a decimal result to a whole number, or fitting a value into a smaller-capacity variable.

Parsing - Whenever handling user input via `Scanner.nextLine()` that needs to become a number.

Why only this way - why not another way?

Why is widening automatic but narrowing isn't -

- * Widening is always safe - a double can represent any value an int can, with no data loss. Java allows this silently since nothing bad can happen.
- * Narrowing can lose data (decimal truncation, or overflow if the value is too big for the smaller type). Java forces you to write `(int)` explicitly as a way of saying "Yes, I understand I might lose data, do it anyway". This prevents accidental data loss from being silently swallowed by the compiler.

Why does `(int) 9.78` give 9, not 10 (no rounding)?

Casting simply truncates - it chops off everything after the decimal point. It is not the same as rounding. If you want rounding, you need `Math.round()`:

`int rounded = Math.round(9.78f);`

Why can't we ^{directly} assign a String to an int (int x = "25"; fails)?

Strings and numbers are stored completely differently in memory (string is a sequence of characters/object "int" is raw binary number).

→ There's no automatic conversion path the compiler can safely assume - should "25" become 25? what about

"twenty-five"? Java requires you to explicitly call Integer.parseInt() so you decide how the conversion should be handled (and can catch errors if the string isn't a valid number).

Real-life Example:

Like pouring a large bottle of water (double) into a small cup (int) - some water spills (data lost).

Pouring a small cup into a large bottle (int to double) - nothing is lost, fits easily.